Understanding Negative Binomial Regression

1 Introduction: The Problem of Overdispersion

Negative binomial regression is a predictive statistical model used specifically for **count data** (0, 1, 2, 3...). It is often used as an alternative to Poisson regression.

In a standard Poisson model, a strict mathematical assumption is made: the mean (average) of the data must equal its variance.

$$E(Y) = Var(Y) = \mu$$

However, in the real world, variance is often much larger than the mean—a phenomenon called **overdispersion**. If Poisson regression is used on overdispersed data, it underestimates standard errors. Negative binomial regression solves this by introducing a dispersion parameter (α) to account for the extra spread:

$$Var(Y) = \mu + \alpha\mu^2$$

Because $\alpha > 0$, the variance will always be greater than the mean.

2 The Mathematical Foundation

To understand how the algorithm works "under the hood," we look at **Maximum Likelihood Estimation (MLE)**.

2.1 1. The Link Function

To ensure the model never predicts a negative count, the algorithm predicts the natural logarithm of the expected count (μ):

$$\ln(\mu) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k$$

Exponents are used to find the actual expected count:

$$\mu = e^{\beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k}$$

2.2 2. The Probability Formula

The Negative Binomial probability mass function calculates the probability of observing exactly y events, given the predicted mean (μ) and dispersion parameter (α):

$$P(Y = y) = \frac{\Gamma(y + \frac{1}{\alpha})}{\Gamma(y+1)\Gamma(\frac{1}{\alpha})} \left(\frac{1}{1+\alpha\mu}\right)^{\frac{1}{\alpha}} \left(\frac{\alpha\mu}{1+\alpha\mu}\right)^{y}$$

2.3 3. The Likelihood and Log-Likelihood

The algorithm finds the weights (β) and dispersion (α) that make the entire dataset highly probable by multiplying individual probabilities (the Likelihood function):

$$L(\beta, \alpha) = \prod_{i=1}^{n} P(y_i \mid \mu_i, \alpha)$$

To avoid computer underflow from multiplying tiny fractions, we take the natural logarithm, converting the product into a sum (the Log-Likelihood function):

$$LL(\beta, \alpha) = \sum_{i=1}^{n} \ln(P(y_i \mid \mu_i, \alpha))$$

The computer uses calculus (e.g., the Newton-Raphson method) to iteratively find the parameters that maximize this equation.

3 Understanding the Independent Variables (X_k)

In the formula $\ln(\mu) = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k$:

- X represents an input variable (e.g., patient age, income, traffic volume).
- **Subscripts (1, 2...)** distinguish one variable from another.
- k is the total number of predictors in the model.

Every X variable is paired with a β **(beta coefficient)**, which is the weight or multiplier the algorithm calculates. The β tells the model how strongly that specific X variable impacts the outcome.

---

4 Summary: The Per-Observation Process

User's Question:
"Let me try to understand what you said. So for each observation, I have a value for each of $X_1, X_2 \ldots X_k$. Given all the β values, I get a predicated mean and the distribution and the likelihood to getting the Y value for that observation. Is this correct?"

Answer:
Yes, exactly! You have completely nailed the concept. That is precisely how the process works on a per-observation basis. Here is the micro-level process summarized:

1. **The Inputs:** You take a single observation (e.g., a patient who is 65 years old for X_1, has 3 conditions for X_2, etc.).
2. **The Predicted Mean:** The algorithm multiplies those X values by its current "guesses" for the β weights to output a predicted mean (μ) specifically for that patient.
3. **The Distribution and Likelihood:** Using the predicted mean (μ) **plus the dispersion parameter (α)**, the algorithm defines the Negative Binomial distribution curve for that patient. It then checks the patient's actual count (the Y value) against that curve to calculate the exact probability (likelihood) of that happening.

To finish the job, the algorithm simply runs that exact sequence for **every single observation** in your dataset. It multiplies all of those individual likelihoods together (or adds their logarithms) to get the Total Log-Likelihood, tweaking the β and α values until it hits the absolute highest probability possible.